

11-字符串专题

11-字符串专题

一、本阶段目标

字符串是 CSP / C++ 中非常常见的基础专题。

这一章的目标是：

- 掌握 `string` 的基本使用
- 理解字符和字符串的区别
- 掌握字符串遍历、拼接、截取、查找
- 掌握字符与数字之间的转换
- 能处理常见字符串模拟题
- 减少下标越界和格式处理错误

字符串专题的核心是：

字符串不是一个整体，而是一串字符。

二、char 和 string 的区别

1. char

`char` 表示一个字符。

例如：

```
char c='a';
```

注意：

`char` 用单引号。

2. string

`string` 表示一串字符。

例如：

```
string s="abc";
```

注意：

```
string 用双引号。
```

3. 常见错误

错误：

```
char c="a";
```

正确：

```
char c='a';
```

错误：

```
string s='a';
```

正确：

```
string s="a";
```

三、string 的定义和输入

1. 定义

```
string s;
```

2. 输入不含空格的字符串

```
cin>>s;
```

例如输入：

```
hello
```

可以正常读入。

3. 输入整行字符串

如果字符串中包含空格，要用：

```
getline(cin,s);
```

例如：

```
hello world
```

4. cin 和 getline 混用

如果前面用了：

```
cin>>n;
```

后面马上用：

```
getline(cin,s);
```

可能会读到上一行残留的换行符。

常见处理：

```
cin.ignore();  
getline(cin,s);
```

四、字符串长度

1. size

```
s.size()
```

表示字符串长度。

2. length

```
s.length()
```

和 `size()` 基本一样。

3. 示例

```
string s="abc";  
  
cout<<s.size();
```

输出：

```
3
```

五、访问字符串中的字符

1. 下标访问

```
s[i]
```

例如：

```
string s="abc";  
  
cout<<s[0];
```

输出：

```
a
```

2. 下标范围

如果字符串长度是 `n`：

合法下标是 `0` 到 `n-1`。

所以：

```
s[s.size()]
```

是越界的。

六、遍历字符串

1. 普通 for

```
for(int i=0;i<s.size();i++)
{
    cout<<s[i]<<"\n";
}
```

2. 范围 for

```
for(char c:s)
{
    cout<<c<<"\n";
}
```

3. 什么时候用普通 for?

如果需要知道下标：

用普通 `for`。

4. 什么时候用范围 for?

如果只关心字符本身：

用范围 for。

七、字符串拼接

1. 用 +

```
string a="hello";  
string b="world";  
  
string c=a+b;
```

结果：

helloworld

2. 用 +=

```
string s="abc";  
  
s+="def";
```

结果：

abcdef

3. push_back

如果只加入一个字符：

```
s.push_back('x');
```

4. 常见错误

错误：

```
s.push_back("x");
```

原因：

push_back 只能加入 char，不能加入 string。

正确：

```
s.push_back('x');
```

或者：

```
s+="x";
```

八、字符判断

1. 判断数字

```
if(c>='0' && c<='9')
{
    // 是数字字符
}
```

2. 判断小写字母

```
if(c>='a' && c<='z')
{
    // 是小写字母
}
```

3. 判断大写字母

```
if(c>='A' && c<='Z')
{
    // 是大写字母
}
```

4. 判断字母

```
if((c>='a' && c<='z') || (c>='A' && c<='Z'))
{
    // 是字母
}
```

九、字符和数字转换

1. 字符转数字

```
int x=c-'0';
```

例如：

```
char c='7';
int x=c-'0';
```

结果：

```
x = 7
```

2. 数字转字符

```
char c=x+'0';
```

例如：

```
int x=7;
char c=x+'0';
```


结果：

```
c = '7'
```

3. 常见错误

错误：

```
int x=c;
```

这得到的是 ASCII 编码，不是数字本身。

十、大小写转换

1. 小写转大写

```
c=c-'a'+'A';
```

2. 大写转小写

```
c=c-'A'+'a';
```

3. 示例

```
char c='b';  
  
c=c-'a'+'A';  
  
cout<<c;
```

输出：

```
B
```

十一、substr 截取子串

1. 基本写法

```
s.substr(pos, len)
```

含义：

从 pos 位置开始，截取 len 个字符。

2. 示例

```
string s="abcdef";  
cout<<s.substr(1,3);
```

输出：

bcd

因为：

从下标 1 开始，取 3 个字符。

3. 省略 len

```
s.substr(pos)
```

表示：

从 pos 开始，一直截到末尾。

4. 易错点

```
s.substr(pos, len)
```

第二个参数是：

长度

不是：

结束下标

十二、find 查找

1. 基本写法

```
s.find(t)
```

表示：

在 `s` 中查找子串 `t` 第一次出现的位置。

2. 示例

```
string s="abcdef";  
int pos=s.find("cd");
```

结果：

`pos = 2`

3. 找不到怎么办？

如果找不到，会返回：

`string::npos`

判断写法：

```
if(s.find(t)!=string::npos)
{
    cout<<"找到了";
}
else
{
    cout<<"没找到";
}
```

4. 查找字符

```
int pos=s.find('a');
```

十三、erase 删除

1. 删除一段

```
s.erase(pos, len);
```

含义：

从 pos 开始删除 len 个字符。

2. 示例

```
string s="abcdef";

s.erase(1,3);
```

结果：

aef

十四、insert 插入

1. 插入字符串

```
s.insert(pos,t);
```

表示：

在 `pos` 位置前插入字符串 `t`。

2. 示例

```
string s="abef";  
s.insert(2,"cd");
```

结果：

abcdef

十五、replace 替换

1. 替换一段

```
s.replace(pos,len,t);
```

含义：

从 `pos` 开始，把 `len` 个字符替换成 `t`。

2. 示例

```
string s="abXXef";  
s.replace(2,2,"cd");
```

结果：

abcdef

十六、回文判断

1. 什么是回文？

回文字符串：

正着读和反着读一样。

例如：

aba
abba

2. 双指针判断

```
bool check(string s)
{
    int l=0;
    int r=s.size()-1;

    while(l<r)
    {
        if(s[l]!=s[r])
        {
            return false;
        }

        l++;
        r--;
    }

    return true;
}
```

十七、字符串反转

1.reverse

```
reverse(s.begin(), s.end());
```

需要头文件：

```
#include<algorithm>
```

2. 示例

```
string s="abc";  
  
reverse(s.begin(), s.end());  
  
cout<<s;
```

输出：

```
cba
```

十八、字符串排序

1. 字符排序

```
sort(s.begin(), s.end());
```

例如：

```
string s="cba";  
  
sort(s.begin(), s.end());
```

结果：

```
abc
```

十九、字符串模拟常见题型

1. 字符统计

统计每个字符出现次数：

```
int cnt[26]={0};

for(int i=0;i<s.size();i++)
{
    cnt[s[i]-'a']++;
}
```

2. 数字字符串求和

高精度中常用：

```
int x=s[i]-'0';
```

3. 字符串展开

根据规则，把某些字符区间展开。

例如：

```
a-d
```

可能展开成：

```
abcd
```

4. 格式处理

例如：

```
把日期、编号、表达式等拆开处理。
```


二十、常见错误

1. char 和 string 混用

```
char 用单引号  
string 用双引号
```

2. 下标越界

错误：

```
s[s.size()]
```

3. substr 第二个参数理解错

```
s.substr(pos, len)
```

len 是长度，不是结束位置。

4. find 找不到没有判断

错误：

```
int pos=s.find(t);  
cout<<s[pos];
```

如果没找到，pos 不是合法下标。

5. getline 被换行符影响

cin>> 后面接 getline 时，要注意：

```
cin.ignore();
```

二十一、本专题做过的题

题号	题目	类型	题面简述	对应知识点
P1098	字符串的展开	字符串模拟	按规则展开字符串中的字符区间	字符处理、模拟
P1203	坏掉的项链	环形字符串	在环形项链中寻找最长可取连续段	字符串遍历、环形处理
P1591	阶乘数码	字符统计 / 高精度	求阶乘结果中某个数字出现次数	高精度、字符统计
P1518	两只塔姆沃斯牛	模拟	模拟两个对象在地图中的移动	字符矩阵、方向模拟
P1210	回文检测	字符串处理	判断处理后的字符串是否为回文	回文、字符过滤

二十二、本章最重要的结论

字符串题的核心是：

把字符串拆成一个个字符处理。

遇到字符串题，先问自己：

我要处理字符？
我要处理子串？
我要查找？
我要替换？
我要统计？
我要模拟规则？

然后选择对应工具：

遍历：for
截取：substr
查找：find
拼接：+= / push_back
反转：reverse
排序：sort